Sentinel — In-Database Real-Time Fraud Detection System

Database Management Systems (CS/AI/AIM/DS 202) — Mini Project Report

Institution: ICFAI Foundation for Higher Education, Faculty of Science & Technology, Hyderabad

Programme: B.Tech CSE (AI & ML), Semester III, A.Y. 2026-27

Team: Sai Haresh Anand S · Seshivardhini

Repository: https://github.com/SAI-HARISH2007/sentinel-sql-progress **Database:** MySQL 8.4 (Docker)

Abstract

Fraud checks usually live in application code: hard-coded thresholds, a gap between commit and check, and a separate copy of the data. Sentinel moves the check *into the database*. A trigger on `INSERT INTO transactions` calls a stored procedure that scores the new row against six rules (high amount, off-hours, new device, velocity, impossible travel, merchant risk), each measured against that account's own history, and writes an alert with human-readable reasons. Thresholds live in a table, so tuning is an `UPDATE`.

The report is organised **week by week**, following the course's lab sequence. Each page was added in the week the matching lab was done, and every query result shown was captured by running the repository's own scripts on the finished database.

Week	Lab topic	Adds to the project
1	Introduction to MySQL & database design (ER)	Design before you build.
2	DDL: CREATE, ALTER, DROP, TRUNCATE, RENAME	Create all 13 tables with columns and data types only.
3	Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT, CHECK	Turn the bare tables into a self-defending schema.
4	DML: INSERT, UPDATE, DELETE, SELECT	Put data in.
5	Functions: number, aggregate, string, date	Use every function family on real data, and use them to build the first three pieces of fraud logic by hand: the z-score (Rule 1), the hour of day (Rule 2) and the distance and speed between cities (Rule 5).
6	Operators: arithmetic, comparison, logical, special, set	Every operator family on real data, ending in a deliberately naive "first-cut fraud screen" built from fixed thresholds, which shows *why* the later weeks compare each account with its own history.
7	Joins: inner, outer, natural, self	Combine tables.
8	GROUP BY, HAVING, ORDER BY — and subqueries	Aggregate per group and filter groups.
9	Views and indexes (plus DCL: roles, GRANT, REVOKE)	Package the eight weeks of queries as named, reusable views, speed up the access paths with measured indexes, and lock the data down with roles.
10	Stored procedures, triggers — and transactions	The finale: the same six rules, now as one stored procedure that a trigger runs on every INSERT.

Week	Lab topic	Adds to the project
11	Testing, documentation and the demo	Prove it works, write it up, rehearse it.

Week 1 — Introduction to MySQL & database design (ER)

Project README · Week 2 →

Lab: Handout lab list **#1 — Introduction to MySQL and ER Diagram** · lab manual **Experiment 0/1 — Introduction to SQL & MySQL and Database Design Theory lectures this lines up with:** 1–9 (introduction to databases, ER modelling, ER → relational design)

Design before you build. You create the empty `sentinel` database and, more importantly, write down the problem, the requirements, the ER diagram and the rules that turn that diagram into tables. No table exists yet.

What was added

File	What it is
`sql/01_create_database.sql`	`DROP/CREATE DATABASE sentinel`, utf8mb4. Every later week starts from this.
`docs/problem-statement.md`	Handout steps 1-4: title, problem, objectives, functional + non-functional requirements, the six fraud signals.
`docs/er-diagram.md` · `er-diagram.dbml`	The ER diagram (Mermaid, renders on GitHub) and DBML for dbdiagram.io. **Generated from the real schema**, never hand-edited.
`docs/er-to-relational.md`	Which ER-to-tables rule produced which table (strong entity, 1:N, M:N, 1:1, participation, derived attribute).
`demo/01_mysql_basics.sql`	`SHOW DATABASES`, `USE`, `SHOW TABLES`, `VERSION()`: the commands in the lab manual's intro page.

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- `SHOW DATABASES` lists `sentinel`; `SHOW TABLES` is empty (tables arrive in Week 2).
- Character set `utf8mb4`, collation `utf8mb4_unicode_ci`.

Concepts you must be able to explain (viva)

- **DBMS vs file system.** A DBMS adds integrity rules, concurrent access control, security and data independence that plain files lack.
- **The five parts of SQL.** DDL (structure) · DML (change data) · DRL (SELECT) · TCL (COMMIT/ROLLBACK) · DCL (GRANT/REVOKE). The lab manual lists exactly these five.
- **ER vocabulary.** Entity, entity set, attribute (simple/composite/multivalued/derived), relationship, cardinality (1:1, 1:N, M:N), participation (total/partial), weak entity.
- **ER → relational.** Read `docs/er-to-relational.md` twice. Each row is a Part B/C exam answer.
- **Why surrogate keys.** `email`, `fingerprint_hash` are candidate keys; a small `INT` id is the primary key because it is compact in every foreign key and index.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout steps 1-5 (title, problem, objectives, requirements, ER model).

Results (captured from the running database)

Evidence: Database and character set

Script: `week-01-intro-er-design/demo/01_mysql_basics.sql`

```
+--------------+
| mysql_version |
+--------------+
| 8.4.11        |
+--------------+
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sentinel           |
| sys                |
+--------------------+
+-----------------+
| current_database |
+-----------------+
| sentinel         |
+-----------------+
+------------------------+
| Tables_in_sentinel     |
+------------------------+
| account_devices        |
| accounts               |
| alerts                 |
| audit_log              |
| behaviour_baselines    |
| devices                |
| locations              |
| login_history          |
... (22 more lines)
```

Week 2 — DDL: CREATE, ALTER, DROP, TRUNCATE, RENAME

Lab: Handout lab list **#2 — Data Definition Language** (CREATE, ALTER, DROP, TRUNCATE, RENAME) · lab manual **"Experiment 2: DDL COMMANDS"** (its index calls it Experiment 1) **Theory lectures this lines up with:** 12–17 (basic SQL: DDL)

Create all 13 tables with columns and data types only. Four things are created deliberately wrong so the ALTER / RENAME commands have real work to do. No keys, no defaults: that is next week.

What was added

File	What it is
`sql/01_create_tables.sql`	13 `CREATE TABLE` statements: columns and types, nothing else.
`sql/02_alter_tables.sql`	`ADD COLUMN`, `RENAME COLUMN`, `MODIFY`, `DROP COLUMN`, `RENAME TABLE`, each fixing one deliberate flaw.
`demo/ddl_tour.sql`	Every DDL command from the manual on a throw-away copy (`CREATE TABLE … LIKE`), cleaned up afterwards.
`lab-practice/ddl_employee.sql`	**The lab manual's 9-question DDL assignment (EMPLOYEE table), solved in MySQL**, in its own `lab_practice` database.

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- 13 tables (`SHOW TABLES`). `DESCRIBE users` has `date_of_birth` right after `phone`, and the `Key` column is **empty** for every table.
- `devices.fingerprint_hash` is `varchar(64)`; `merchants.merchant_name` is `varchar(200)`; `fraud_alerts` is now called `alerts` and has no `notes` column.

Concepts you must be able to explain (viva)

- **Data types.** `INT`, `BIGINT`, `DECIMAL(p,s)` (exact: use for money, never FLOAT), `CHAR` vs `VARCHAR`, `TEXT`, `ENUM`, `JSON`, `TIMESTAMP`, `BOOLEAN` (= `TINYINT(1)`).
- **ALTER TABLE.** `ADD`, `MODIFY` (change type/width), `RENAME COLUMN`, `DROP COLUMN`.
- **DELETE vs TRUNCATE vs DROP.** DELETE: chosen rows, can roll back, fires triggers. TRUNCATE: all rows, DDL, resets AUTO_INCREMENT, no rollback. DROP: rows *and* table.
- **DDL commits implicitly.** You cannot ROLLBACK a CREATE/ALTER/DROP/TRUNCATE.
- **Why bare tables first?.** A table without keys is legal but unsafe: it would accept duplicate users and orphan transactions. Week 3 shows the fix.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout step 7 (SQL implementation: DDL).

Results (captured from the running database)

Evidence: Every DDL command on a throw-away table

Script: `week-02-ddl/demo/ddl_tour.sql`

```
+---------------+-------------+------+-----+-------------------+-------------------+
| Field         | Type        | Null | Key | Default           | Extra             |
+---------------+-------------+------+-----+-------------------+-------------------+
| user_id       | int         | NO   | PRI | NULL              | auto_increment    |
| full_name     | varchar(150) | NO   |     | NULL              |                   |
| email         | varchar(255) | NO   | UNI | NULL              |                   |
| phone         | varchar(20)  | YES  |     | NULL              |                   |
| date_of_birth | date        | YES  |     | NULL              |                   |
| created_at    | timestamp   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+-------------+------+-----+-------------------+-------------------+
+---------------+-------------+------+-----+-------------------+-------------------+
| Field         | Type        | Null | Key | Default           | Extra             |
+---------------+-------------+------+-----+-------------------+-------------------+
| user_id       | int         | NO   | PRI | NULL              | auto_increment    |
| full_name     | varchar(150) | NO   |     | NULL              |                   |
| email         | varchar(255) | NO   | UNI | NULL              |                   |
| phone         | varchar(20)  | YES  |     | NULL              |                   |
| date_of_birth | date        | YES  |     | NULL              |                   |
| created_at    | timestamp   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+-------------+------+-----+-------------------+-------------------+
+------------+-------------------
```